

Lecture 16

Quantization for AI/NPU Compiler

INT8/INT4 requantization

Accumulator width and saturation

MLIR quant dialect → TOSA rescale → NPU qkernel

오늘의 위치

11 강 : legal op set

12 강 : bufferization

13 강 : memory space/layout

14 강 : vector tensorization

15 강 : transform schedule

16 강 : numeric contract for integer NPU

Learning Objectives

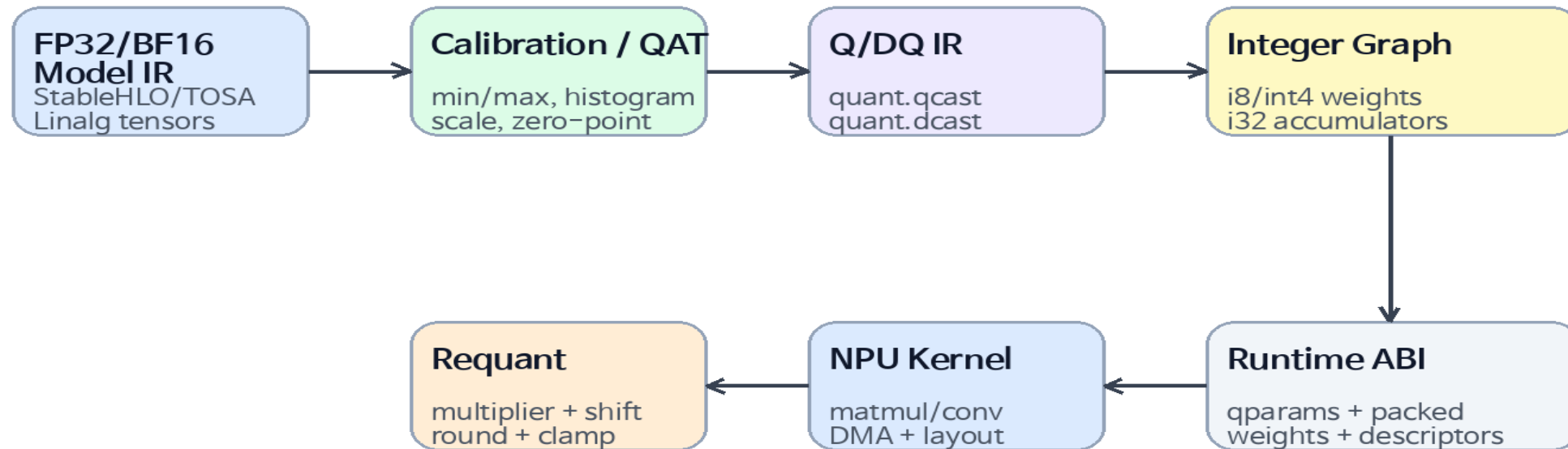
Uniform quantization 수식을 compiler contract 로 해석
!quant.uniform, qcast/dcast/scast 읽기
INT8 MatMul accumulator 와 requantization 설계
INT4 weight packing 과 group scale policy 정의
negative test 로 overflow/rounding/layout 오류 잡기

왜 NPU Compiler 에서 Quantization 이 중요한가 ?

TOPS/W 와 bandwidth 는 낮은 bit-width 에서 크게 개선
정확도는 scale/zero-point/rounding/saturation 에 민감
compiler 가 qparams, layout, DMA, ABI 를 함께 관리해야 함
Numeric bug 는 functional test 에서는 늦게 드러남

Quantization Compiler Flow

Quantization Compiler Flow for NPU



Key compiler contract: keep floating semantics long enough to choose qparams, then lower to legal integer kernels with explicit rounding and saturation.

Uniform Quantization

```
q = clamp(round(x / scale) + zero_point)
x_hat = (q - zero_point) * scale
scale: real-space step size
zero_point: exact real zero code
```

Fixed-point vs Affine

Fixed-point: signed scaled integer

Affine: shifted integer space with zero_point

Conv/MatMul zero padding 때문에 zero exactness 가 중요

NPU 는 affine operand 를 scaled operand 로 보정 후 MAC

MLIR Quant Dialect

!quant.uniform encodes stored type, expressed type, scale, zero point

Per-layer, per-channel/per-axis, sub-channel/blockwise variants

qcast: expressed $\rightarrow$ quantized

dcast: quantized $\rightarrow$ expressed

scast: quantized type $\langle - \rangle$ integer storage type

Example: !quant.uniform

```
!quant.uniform<i8:f32, 0.03125:-3>  
tensor<64x!quant.uniform<i8:f32, 0.03125:-3>>  
tensor<64x128x!quant.uniform<i8:f32:1,{0.02,0.03,...}>>
```

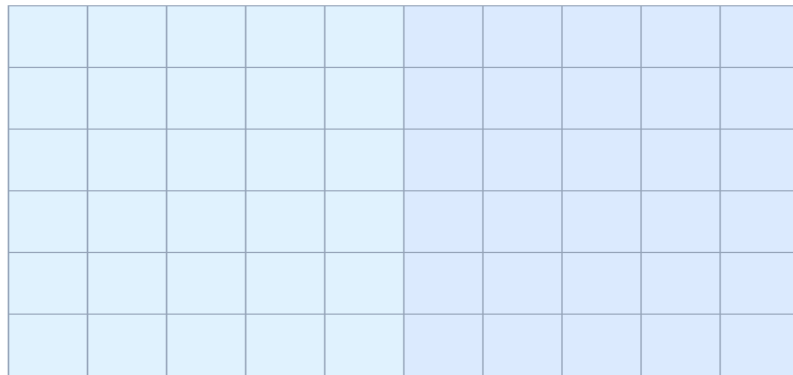
Q/DQ Boundary

qcast/dcast pair 는 quantization intent 를 IR 에 보존
integerizable subgraph 를 찾는 marker 역할
fallback boundary 와 accuracy debug 가 쉬워짐
너무 일찍 integer lowering 하면 fusion/layout 판단이 어려움

Per-tensor vs Per-channel

Per-tensor vs Per-channel Quantization

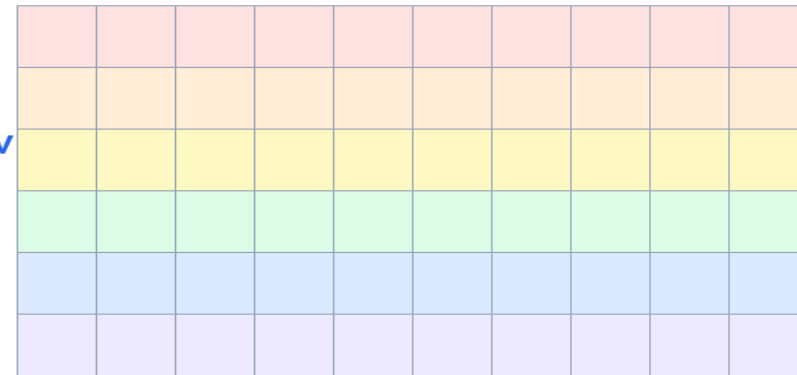
Activation tensor: one scale



scale_a = 0.031, zero_a = -3

MatMul / Conv

Weight tensor: per-output-channel scale



scale_w [0]

scale_w [1]

scale_w [2]

scale_w [3]

scale_w [4]

scale_w [5]

Per-channel requant: multiplier[channel] and shift[channel] follow output channel / filter dimension.
Compiler task: verify channel axis, length, layout transform, and runtime descriptor metadata.

Blockwise / Group Quantization

INT4 weight-only quantization 에서 자주 사용

group_size = 32/64/128 등

scale layout = per output channel x per K group

Compiler must align group boundary with packed weight layout

INT8 MatMul Equation

```
acc[m,n] =  $\sum_k$  (qa[m,k]-za) * (qw[k,n]-zw[n])  
real_y = sa * sw[n] * acc + bias_real[n]  
qy = clamp(round(real_y / sy) + zy)
```

Bias Scale Contract

`bias_i32[n]` usually represents $\text{bias_real} / (\text{sa} * \text{sw}[n])$

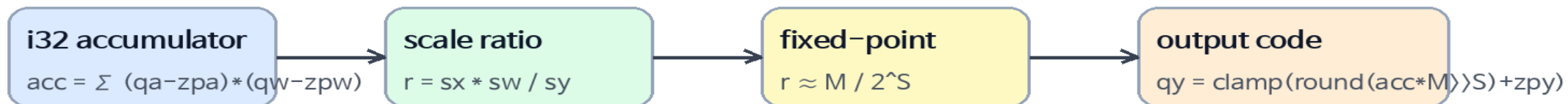
Per-channel weight scale $\rightarrow$ per-channel bias scale

Verifier: bias dtype, scale, channel axis, shape

Mismatch causes silent accuracy loss

Requantization

Requantization: i32 accumulator $\rightarrow$ i8 output



```
real_y = sy * (qy - zpy)
real_acc = sx * sw * acc
qy = saturate_i8( round(acc * sx * sw / sy) + zpy )
fixed-point: qy = saturate_i8( rounding_shift(acc * M, S) + zpy )
```

NPU HW contract: rounding mode, saturation range, per-channel multiplier source, and accumulator bit-width must be explicit in IR/AB

TOSA Rescale

tosa.rescale represents integer multiply, add, shift

Attributes include scale32, rounding_mode, per_channel, input/output unsigned

Operands include multiplier, shift, input_zp, output_zp

Good bridge from tensor IR to integer NPU kernels

Rounding Modes

RNE / round-to-nearest-even

away-from-zero or framework-specific rounding

two-stage rounding for some framework compatibility

Changing rounding can shift quantized outputs by 1 LSB

Saturation Semantics

i8 range: $[-128, 127]$ or restricted $[-127, 127]$

u8 range: $[0, 255]$

int4 signed: $[-8, 7]$ or symmetric $[-7, 7]$

Clamp range must match target kernel and quant type

Accumulator Width

Accumulator Width Analysis

case	operand range	max product	K safe in i32
int8 sym	$[-127, 127] \times [-127, 127]$	16,129	~133k
uint8 affine	$[-255, 255] \times [-255, 255]$	65,025	~33k
int4 sym	$[-7, 7] \times [-7, 7]$	49	~43M
int4 unsigned	$[0, 15] \times [0, 15]$	225	~9.5M

$\text{Required bits} \approx 1 + \text{ceil}(\log_2(K * \text{max_abs_a} * \text{max_abs_w} + \text{bias_margin} + 1))$

NPU compiler must reject/retile/fallback when K, zero-point correction, or bias add can overflow target accumulator.

주의: 표는 worst-case bound입니다. 실제 모델은 calibration/statistics로 더 작을 수 있지만 verifier는 worst-case 또는 profiled contract를 명확히 해야 합니다.

Zero-point Correction

$\Sigma(qa - za)(qw - zw)$ expands into four terms

Need row sums / column sums when za or zw are nonzero

Symmetric weights simplify correction

Precompute weight sums when possible

Symmetric vs Asymmetric

Symmetric: zero_point=0, simpler MAC path

Asymmetric: better range utilization for activations

Weights often symmetric per-channel

Activations often affine per-tensor

INT4 Overview

INT4 Weight Quantization and Packing

Logical int4 weights



Packed bytes in DRAM/SRAM



Compiler checks:

- signedness: s4 vs u4
- nibble order: low/high
- group size: e.g. 32/64/128
- scale granularity: per-channel or per-group
- K alignment for tensor core
- unpack path vs native int4 MAC

Pseudo IR:

```
npu_kernel.qmatmul {lhs_type="i8", rhs_type="s4",  
  rhs_packing="two_per_byte_low_first", group_size=64,  
  scale_layout="per_out_channel_per_group"}
```

INT4 Packing Contract

signedness: s4 / u4

nibble order: low-first or high-first

K alignment: native tile K multiple

group_size and qparams layout

DMA burst/padding requirements

npu_kernel.qmatmul Contract

lhs_type, rhs_type, acc_type

lhs_scale, rhs_scale, output_scale

zero points and per-channel axis

multiplier/shift tensors

packing, rounding, saturation

Quantization Pipeline

stats import -> qparams assignment
insert/recognize QDQ
propagate/canonicalize QDQ
integerize legal subgraphs
materialize requant
pack weights and emit runtime descriptors

Where to Lower?

Before bufferization: better graph rewrite and fusion

After layout propagation: qparams axis is stable

Before runtime ABI: weight packing and descriptors

Do not mix semantic QDQ and physical storage too early

FileCheck Strategy

CHECK presence of qmatmul and rescale

CHECK-NOT unexpected dequantize in integer kernel

CHECK multiplier/shift rank and per_channel attr

negative test unsupported group size / overflow

Numerical Golden Tests

Compare against FP reference with tolerance

Use edge values: min/max, zero, saturation boundary

Test per-channel qparams and layout-transposed tensors

Lock rounding mode exactly

Exercise A

Given min/max, compute scale and zero_point

Quantize sample vector and dequantize it

Measure max absolute quantization error

Explain impact on layer output

Exercise B

Annotate !quant.uniform types

Insert qcast/dcast for activation and weights

Use scast to expose integer storage

Write FileCheck lines

Exercise C

For INT8 MatMul $K=4096$, compute worst-case accumulator bound

Choose i32 or i48 accumulator

Decide split- K /fallback policy

Document verifier rule

Exercise D

Design INT4 weight packing policy
Define group_size and scale layout
Write npu_kernel.qmatmul attributes
Create negative test for misaligned K

Common Mistakes

Using per-channel scale after layout transpose without remapping axis

Dropping zero-point correction

Unspecified rounding mode

Bias scale mismatch

Assuming int4 packing order from byte dump

Takeaways

Quantization is a numeric ABI, not only dtype conversion

MLIR quant types preserve expressed/stored mapping

tosa.rescale models integer requantization

Accumulator and saturation must be verified

INT4 requires packing + qparams co-design

References

MLIR Quant Dialect: <https://mlir.llvm.org/docs/Dialects/QuantDialect/>

MLIR Quantization Design: <https://mlir.llvm.org/docs/Quantization/>

MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/>

MLIR Passes: <https://mlir.llvm.org/docs/Passes/>